

JavaScript Modules

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Why Modules?

- Split code into separate files, each with its own **scope**
- **Export** things from one file, **import** them in another
- Easier to organize, reuse, and reason about

```
// math.js
export function add(a, b) { return a + b; }

// main.js
import { add } from "./math.js";
```

Before Modules: Multiple Scripts

- The old way to split browser code across files:

```
<script src="helpers.js"></script>  
<script src="app.js"></script>
```

- Run in **document order**: `app.js` must come after what it uses
- All share **one global scope** (`window`), easy to **collide**
- Modules fix this: per-file scope + explicit `import` / `export`

Named Exports

- Many per module, imported by exact name
- Inline with the declaration, or as a list

```
export const PI = 3.14159;  
export function add(a, b) {  
  return a + b;  
}  
  
const x = 1, y = 2;  
function multiply(a, b) { return a * b; }  
export { x, y, multiply };
```

- Rename on export: `export { secret as answer };`

Default Export

- Each module may have **one** default export

```
export default function greet(name) {  
  console.log("Hello, " + name);  
}
```

- Can also default-export a value or class

```
const config = { debug: true };  
export default config;  
// export default class Animal {}
```

Importing Named and Default

```
import { PI, add } from "./math.js";      // named
import { add as sum } from "./math.js";   // renamed
import greet from "./greet.js";          // default
import greet, { PI, add } from "./mod.js"; // both
```

- Named imports need the exact exported names in braces
- The default import name is your choice
- The braces are **not** destructuring; it is import syntax

Namespace and Side-Effect Imports

- **Namespace:** collect all named exports into one object

```
import * as math from "./math.js";  
// math.PI, math.add(...) (default is math.default)
```

- **Side-effect only:** run a module for its effects, no bindings

```
import "./setup.js";
```

Re-Exporting

- Export bindings straight from another module

```
export { add, PI } from "./math.js";  
export { add as sum } from "./math.js";  
export * from "./math.js";           // all named exports
```

- Mix local and re-exported, the "barrel file" pattern

```
export const version = "1.0.0";  
export * from "./math.js";
```


Top-Level Only

- Static `import` / `export` must be at the **top level**
- Not allowed inside functions, `if`, loops, etc.

```
// valid
import { add } from "./math.js";

// invalid
if (true) {
  import { add } from "./math.js";
}
```

- Structure is known before code runs, that's what makes them **static**

Imports Are Hoisted

```
console.log(add(2, 3)); // 5 - works!  
import { add } from "./math.js";
```

- The whole module graph is resolved **before** any code runs
- So import position does not matter, but put them at the top anyway
- This is also why the path must be a literal string, not built at runtime

In the Browser: `type="module"`

- Mark a `<script>` as a module to use `import` / `export`

```
<script type="module" src="main.js"></script>
```

```
<script type="module">  
  import { add } from "./math.js";  
  console.log(add(5, 7));  
</script>
```

- Module scripts are **deferred** automatically (run after HTML parsing)
- The `.js` extension in the path is **required**

In Node: Marking a File as an ES Module

- Node supports both ES modules and CommonJS (`require`)
- A file is an ES module if:
 - The nearest `package.json` has `"type": "module"`, or
 - The file uses the `.mjs` extension
- Once it's an ES module, the syntax matches the browser
- A `.cjs` file stays CommonJS even inside a `"type": "module"` package

CommonJS: The Older System

```
// utils.cjs
exports.add = function (a, b) { return a + b; };
// or: module.exports = { add };

// index.cjs
const utils = require("./utils.cjs");
console.log(utils.add(2, 3));
```

- `require` is **synchronous** and resolved at **runtime**
- Still everywhere in older Node code; you will read it

Reaching CommonJS From an ES Module

- There is no `require` in an ES module, make one

```
import { createRequire } from "node:module";
const require = createRequire(import.meta.url);

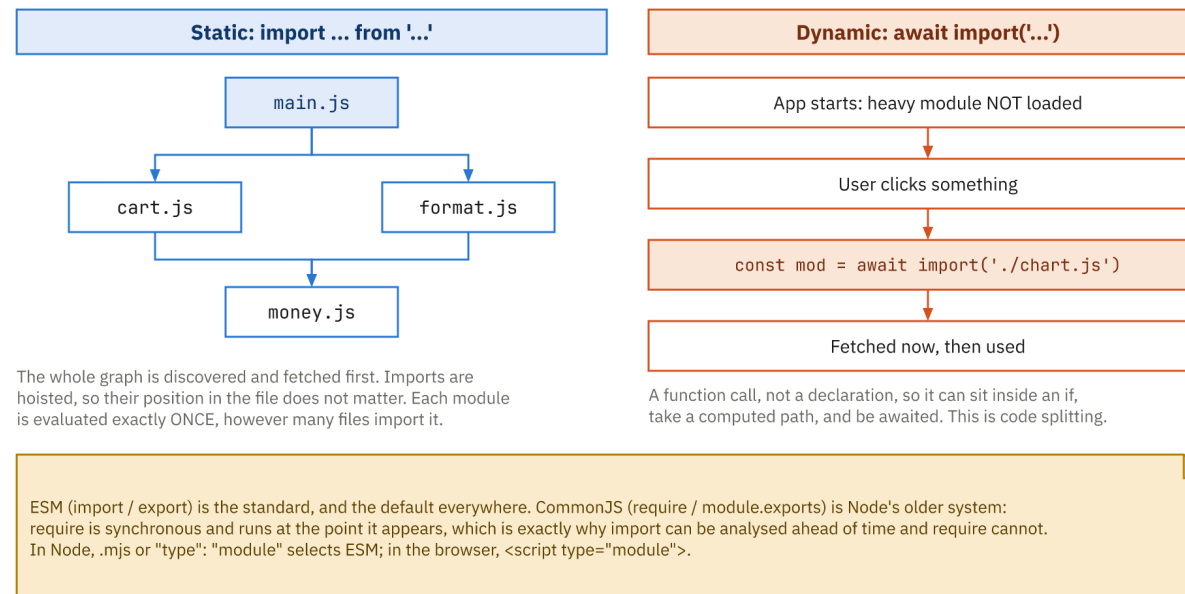
const legacy = require("./legacy-utils.cjs");
console.log(legacy.add(2, 3));
```

	ES modules	CommonJS
Syntax	<code>import</code> / <code>export</code>	<code>require</code> / <code>module.exports</code>
Resolved	before run (static)	at runtime
Top-level <code>await</code>	yes	no

The Module Graph

The Module Graph

Static imports are resolved before any of your code runs. Dynamic `import()` happens later, and returns a promise.



- `import` is resolved before any of your code runs
- Each module is evaluated exactly **once**, however many files import it